`MockMvc` is a utility class that lets you test web endpoints without loading a server like Tomcat. Such a test is naturally lighter than the one we wrote in the previous section, where an embedded server was needed to run the test.

Slice tests run against an application context containing only the parts of the configuration requested by that application slice. In the case of collaborating beans outside the slice, such as the `BookService` class, we use mocks.

Mocks created with the `@MockBean` annotation are different from standard mocks (for example, those created with Mockito) since the class is not only mocked, but the mock is also included in the application context. Whenever the context is asked to autowire that bean, it automatically injects the mock rather than the actual implementation.

3.4.4 Testing the JSON serialization with @JsonTest

The `Book` objects returned by the methods in `BookController` are parsed into JSON objects. By default, Spring Boot automatically configures the Jackson library to parse Java objects into JSON (*serialization*) and vice versa (*deserialization*).

Using the `@JsonTest` annotation, you can test JSON serialization and deserialization for your domain objects. `@JsonTest` loads a Spring application context and auto-configures the JSON mappers for the specific library in use (by default, it's Jackson). Furthermore, it configures the `JacksonTester` utility, which you can use to check that the JSON mapping works as expected, relying on the JsonPath and JSONAssert libraries.

NOTE JsonPath provides expressions you can use to navigate a JSON object and extract data from it. For example, if I wanted to get the `isbn` field from the `Book` object's JSON representation, I